CSCI 2800 — Computer Architecture and Operating Systems (CAOS)
Lab 8 (document version 1.1)

- This lab is due by the **end of your lab session** on Wednesday, November 4, 2026
- This lab is to be completed **individually**; do **not** share your code with anyone else
- Do **not** use AI to assist you; instead, use posted examples, the Discussion Forum, trial-and-error, and your own intelligence
- You **must** show your code and your solutions to a TA, CA, or mentor to receive credit for each checkpoint
- Note that all of your C code **must** successfully compile via `gcc` with no warning messages when the `-Wall` (i.e., warn all) compiler option is used; also use `-Werror`, which treats all warnings as critical errors

For this lab, the goal is to use most of the MIPS instructions and pseudo-instructions listed below. Note that some instructions have "immediate" counterparts, e.g., `add` and `addi`; and the load and store instructions also have counterparts for different sizes (e.g., `lbu` and `sb`). Also note that pseudo-instructions are marked with an asterisk `*`. Instructions/pseudo-instructions are: `add`; `and`; `beq`; `bne`; `j`; `jal`; `jr`; `la*`; `li*`; `lw`; `move*`; `or`; `sll`; `slt`; `srl`; `sw`; and `sub`.

For any of the above instructions that you do not use for this lab, make sure you understand and practice using them elsewhere!

1. **Checkpoint 1:** For the first checkpoint, using MIPS, write a function labeled `tolowerb` that converts one byte of memory to a lowercase letter. Register `$a0` contains the address of the byte to be "downcased"—start by checking to see if the byte is an uppercase letter, and if so, convert it. Either way, use register `$v0` as the return value for your function—i.e., return the resultant byte (whether changed or not) in `$v0`.

   Next, write a function labeled `tolower` that converts an entire character string to lowercase. Register `$a0` contains the address of the *first* byte of the string to be "downcased." Make use of your `tolowerb` function by calling it repeatedly in a loop until you reach the end of the string, which is indicated by a zero byte.

   Test the above thoroughly and use the registers indicated, then move on to the last requirement, which is to make this program interactive. Repeatedly ask the user to input a string by using the appropriate system call. Echo back the "downcased" string; if the user enters `"DONE"`, exit the program.

   Be sure that for your function calls, you preserve all saved registers `$s0-$s7`, the stack pointer `$sp` and the return address `$ra`, as indicated on the MIPS Reference Card.

   Be fully prepared to explain what each MIPS instruction in your code does.

2. **Checkpoint 2:** For the second checkpoint, download the `lab08.s` MIPS code file from the corresponding `lab08` folder. This code is incomplete, so start by determining what the code does and what it should output. Specifically, what does the `bcount` function do?

   To complete the code, first implement the `printb` procedure, which should print one bit to the output based on input parameter `$a1`. (Currently, the `printb` procedure returns immediately by simply executing `jr $ra`.) Specifically, if `$a1` is zero, display the given `zerotxt` string; otherwise, display the given `onetxt` string. For your implementation, write the `printb` procedure such that it has **exactly one** branch instruction.

   Second, expand the `bcount` function to display the line of output shown below.

   ```
   00001111111100000011110000111100 --> '1' bits: 16
   ```

   For your implementation, do **not** use any pseudo-instructions. Replace all pseudo-instructions already present in the given code with MIPS instructions. You can determine how pseudo-instructions are replaced by running the pseudo-instruction in `spim`.

   Finally, to better view the secret message, change `zerotxt` to `'-'` and `onetxt` to `'$'`, then execute the code again.

   To receive credit for this checkpoint, be sure you are able to describe your code and show your program output.

3. **Checkpoint 3: (v1.1)** Be sure you complete Checkpoint 2 before proceeding with this checkpoint.

   Secret messages come in various sizes, not just 32-bit widths. Therefore, for this third checkpoint, extend your work on Checkpoint 2 by adding a *width* parameter to your `bcount` function. Specifically, `$a2` is the width to use to decode the given 32-bit `$a0` parameter. This width will be from 1 to 32 and will specify how many bits to count and display, going left to right (i.e., from high-order bits down to low-order bits), before showing the count and displaying a newline `'\n'` character. Note that a full 32-bit `0x00000000` word will still indicate the end of the data.

   The trickier part is that the entire 32-bit value is always decoded; therefore, consider using the `sll` and `srl` instructions. Below are three examples that illustrate the required behavior. For these examples, the `secret` data are set as shown.

```
secret: .word 0x0ff03c3c, 0xf00ff00f, 0x3c3c0ff0, 0x00000000

[with $a2 set to 32 (default)]
----$$$$$$$$------$$$$----$$$$-- --> '1' bits: 16
$$$$--------$$$$$$$$--------$$$$ --> '1' bits: 16
--$$$$----$$$$------$$$$$$$$---- --> '1' bits: 16

[with $a2 set to 16]
----$$$$$$$$---- --> '1' bits: 8
--$$$$----$$$$-- --> '1' bits: 8
$$$$--------$$$$ --> '1' bits: 8
$$$$--------$$$$ --> '1' bits: 8
--$$$$----$$$$-- --> '1' bits: 8
----$$$$$$$$---- --> '1' bits: 8

[with $a2 set to 11
----$$$$$$$ --> '1' bits: 7
$------$$$$ --> '1' bits: 5
----$$$$-- --> '1' bits: 4
$$$$------- --> '1' bits: 4
-$$$$$$$$-- --> '1' bits: 8
------$$$$ --> '1' bits: 4
--$$$$----$ --> '1' bits: 5
$$$------$$ --> '1' bits: 5
$$$$$$---- --> '1' bits: 6
```

   Once you have completed your implementation, can you decode the `secret2` data in the given `lab08.s` file? In other words, what is the correct width to use to decode these data?

   To receive credit for this checkpoint, be sure you are able to describe your code and show your program output.